

浙江大学

本科实验报告

课程名称:	数字逻辑电路设计
姓 名:	王传宇
学 院:	计算机科学与技术学院
专 业:	计算机科学与技术
邮 箱:	3250102681@zju.edu.cn
QQ 号:	3801249104
电 话:	15118159101
指导教师:	洪奇军
报告日期:	2026 年 4 月 9 日

浙江大学实验报告

课程名称: 数字逻辑设计实验 实验类型: 综合
实验项目名称: 七段数码管译码器设计与 FPGA 验证
学生姓名: 王传宇 学号: 3250102681 同组学生姓名: 齐思航
实验地点: 紫金港东四 509 室 实验日期: 2026 年 4 月 9 日

一、操作方法与实验步骤

1. 设计实现 MC14495 译码器

- 模块功能分析

本模块实现 4 位 BCD 至七段码译码, 功能参照 MC14495 芯片。
输入 D3~D0, 锁存使能 LE 低有效, 小数点 point 高有效。输出 a~g、p 均为低有效。

- Verilog 代码编写

新建 MyMC14495.v, 用 reg [3:0] latched_data 实现锁存:
LE=1 保持, LE=0 更新。组合逻辑 case 译码, 段码值取反以满足
低有效输出, $p = \sim point$ 。

- 仿真测试平台搭建

新建 MyMC14495_tb.v, 例化待测模块。激励中先令 LE 为低, 遍历输入 0~15, 再翻转 point, 最后拉高 LE 验证锁存。仿真时长
200 ns。

- 仿真运行与波形分析

Vivado 中运行行为仿真, 观测 D3~D0、LE、point 及输出波形,
确认译码正确、锁存有效。

2. 顶层模块封装与下板验证

- 顶层模块设计

新建 DispNum.v 作为顶层。端口映射: SW[3:0]→数据输入,
SW[7:4]→AN 位选, BTN[0]→LE, BTN[1]→point, BTN_X 输出恒
0。例化 MyMC14495, 用 assign SEGMENT =

{p,g,f,e,d,c,b,a} 拼接段码。

- 引脚约束文件编写

编写 `DispNum_constraint.xdc`，配置 `SW`、`BTN`、`AN`、`SEGMENT` 各引脚位置及 `I/O` 标准。针对布局错误，加入 `CLOCK_DEDICATED_ROUTE FALSE` 约束禁用 `BTN[0]` 的时钟走线。

- 综合、实现与比特流生成

设置 `DispNum` 为顶层，添加约束文件。依次综合、实现，修正时序警告后生成比特流。

- 下板测试操作

下载程序后，调节 `SW[7:4]` 仅点亮左右两数码管。分别设置 `SW[3:0]=0000/1000/1111`，观察显示 `0`、`8`、`F`。测试锁存与小数点功能，记录照片。

3. I LovE YOU decoder 设计

- 设计目标

设计一个 3 输入 7 输出的译码器，输入 `D2~D0` 从 `000~111` 依次对应字符 `I`、`L`、`o`、`v`、`E`、`Y`、`O`、`U`，输出 `a~g` 驱动七段数码管显示对应字母。

- 真值表建立

根据各字母的段位点亮描述，列出真值表如下（输出高电平有效，`0` 亮 `1` 灭）：

D2D1D0	a	b	c	d	e	f	g
0	1	1	1	1	0	0	1
1	1	1	1	0	0	0	1
2	1	1	0	0	0	1	0
3	1	1	0	0	0	1	1
4	0	1	1	0	0	0	0
5	1	0	0	0	1	0	0
6	0	0	0	0	0	0	1
7	1	0	0	0	0	0	1

- 布尔表达式化简

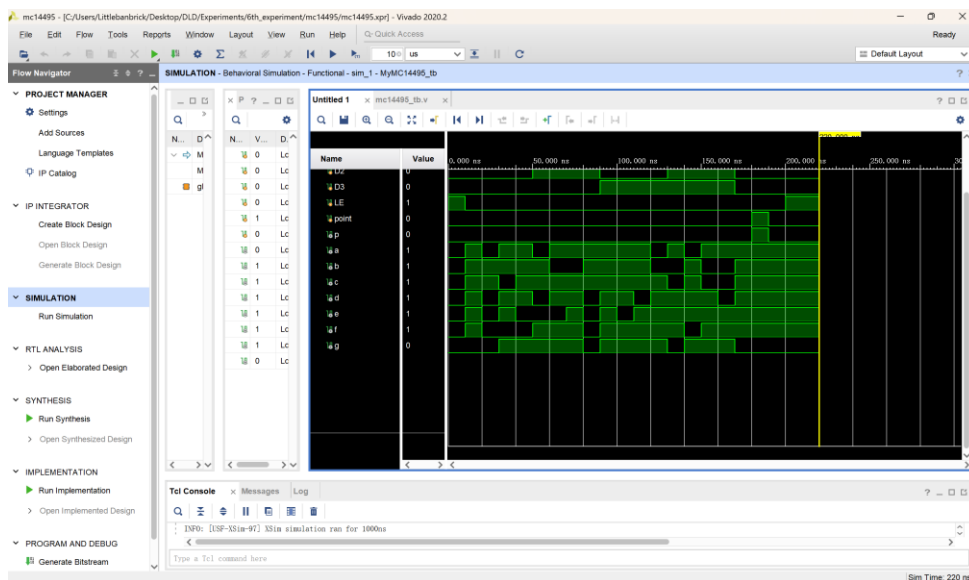
选取输出 `a`、`b`、`c`、`g` 进行化简。采用卡诺图法，`D2` 为高位，`D1 D0` 按格雷码排列。

卡诺图及详细圈项步骤见实验结果部分。

二、实验结果与分析

1. MC14495 模块仿真结果

— 仿真波形截图

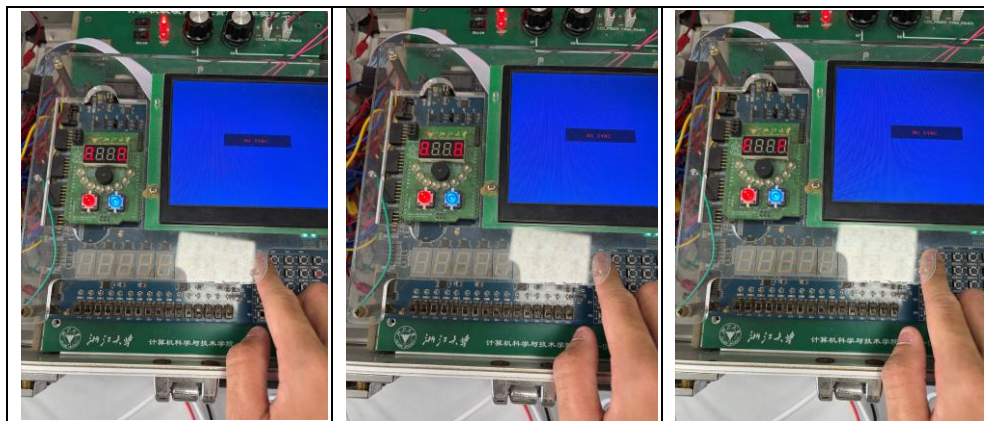


— 波形分析

波形显示 **LE** 为低时，输出随输入变化，译码正确。**LE** 拉高后输出锁存，输入变化无效。**point** 为高时 **p** 输出低，小数点亮。功能完全符合预期。

2. DispNum 顶层下板验证结果

— 硬件测试照片

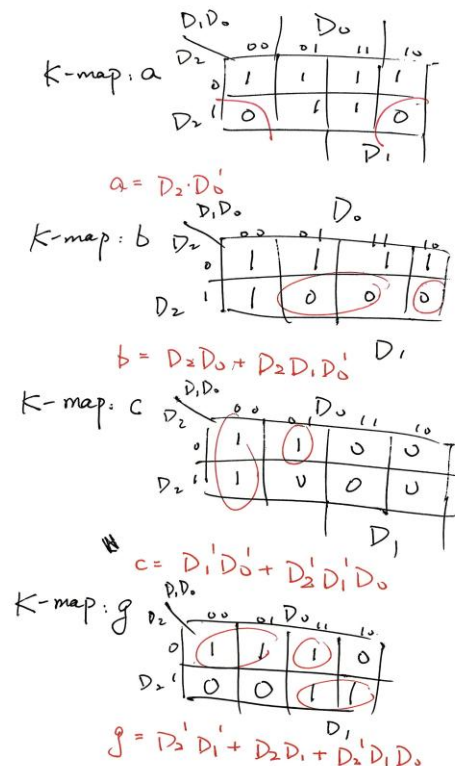


— 功能验证分析

AN 控制正确，仅左右两管点亮。段位显示准确，**0**、**8**、**F** 无缺段。锁存按钮按下显示保持，小数点按钮响应正常。硬件行为与仿真一致。

3. I Love YOU decoder 结果分析

- 化简结果和卡诺图展示



三、讨论和心得

这次实验整体还算顺利，不过还是遇到了两个比较典型的问题。首先是比特流生成时的 **Place 30-574** 报错，工具把普通按钮信号 **BTN[0]** 当成了时钟来处理，导致布线失败。一开始有点懵，后来根据错误提示在约束文件里加了一句 ``set_property CLOCK_DEDICATED_ROUTE FALSE``，问题就解决了。这种工具层面的“误判”确实有点玄学，但吃一堑长一智，以后遇到类似情况也知道怎么处理了。另一个问题是段码极性，一开始没注意输出是低有效，直接用了高有效的码表，下板之后显示完全反了。回头一翻实验要求才发现漏看了关键一句话，改过来就好了。这也提醒自己写代码前先把端口要求看清楚，否则改来改去浪费时间。

经过这次实验，对七段译码器的底层逻辑以及从设计到上板的完整流程更加熟练了。从 **Logisim** 画图到 **Verilog** 代码编写，再到仿真、约束、下板验证，整套流程走下来心里更有底了。总体而言，虽然中间踩了些小坑，但收获还是不小的。

附录：源代码

[mc14495.v]

```
module MyMC14495(

    input D0, D1, D2, D3,      // 4 位二进制输入

    input LE,                  // 锁存使能，低电平有效

    input point,               // 小数点输入，高电平有效

    output reg p,              // 小数点输出，低电平有效

    output reg a, b, c, d, e, f, g // 七段笔画输出，低电平有效
);

    reg [3:0] latched_data;

    // LE 低电平时透明传输，高电平时锁存（因为 LE 低有效）

    always @(*) begin

        if (LE)                // LE=0 时更新，LE=1 时保持

            latched_data = latched_data; // 保持

        else

            latched_data = {D3, D2, D1, D0};

    end

    always @(*) begin

        p = ~point; // 小数点输出低有效，所以对输入取反
```

```
case (latched_data)
```

```
// 以下段码均为取反后的值（低电平有效）
```

```
4'd0: {a,b,c,d,e,f,g} = 7'b00000001; // 显示 0
```

```
4'd1: {a,b,c,d,e,f,g} = 7'b1001111; // 显示 1
```

```
4'd2: {a,b,c,d,e,f,g} = 7'b0010010; // 显示 2
```

```
4'd3: {a,b,c,d,e,f,g} = 7'b0000110; // 显示 3
```

```
4'd4: {a,b,c,d,e,f,g} = 7'b1001100; // 显示 4
```

```
4'd5: {a,b,c,d,e,f,g} = 7'b0100100; // 显示 5
```

```
4'd6: {a,b,c,d,e,f,g} = 7'b0100000; // 显示 6
```

```
4'd7: {a,b,c,d,e,f,g} = 7'b0001111; // 显示 7
```

```
4'd8: {a,b,c,d,e,f,g} = 7'b0000000; // 显示 8
```

```
4'd9: {a,b,c,d,e,f,g} = 7'b0000100; // 显示 9
```

```
4'd10: {a,b,c,d,e,f,g} = 7'b0001000; // 显示 A
```

```
4'd11: {a,b,c,d,e,f,g} = 7'b1100000; // 显示 b
```

```
4'd12: {a,b,c,d,e,f,g} = 7'b0110001; // 显示 C
```

```
4'd13: {a,b,c,d,e,f,g} = 7'b1000010; // 显示 d
```

```
4'd14: {a,b,c,d,e,f,g} = 7'b0110000; // 显示 E
```

```
4'd15: {a,b,c,d,e,f,g} = 7'b0111000; // 显示 F
```

```
default: {a,b,c,d,e,f,g} = 7'b1111111; // 全灭
```

```
endcase
```

```
end
```

```
endmodule
```

[mc14495_tb.v]

```
`timescale 1ns / 1ps

module MyMC14495_tb();

// Inputs

reg D0, D1, D2, D3;

reg LE;

reg point;

// Outputs

wire p;

wire a, b, c, d, e, f, g;

// 实例化被测模块

MyMC14495 uut (

    .D0(D0),

    .D1(D1),

    .D2(D2),

    .D3(D3),

    .LE(LE),

    .point(point),

    .p(p),
```

```
.a(a),  
  
.b(b),  
  
.c(c),  
  
.d(d),  
  
.e(e),  
  
.f(f),  
  
.g(g)  
);  
  
initial begin  
  
    // 初始化  
  
    LE = 1;  
  
    point = 0;  
  
    {D3,D2,D1,D0} = 4'b0000;  
  
  
    #10 LE = 0;  
  
  
    // 测试 0~F  
  
    repeat (16) begin  
  
        #10 {D3,D2,D1,D0} = {D3,D2,D1,D0} + 1;  
  
    end  
  
  
    // 测试小数点
```

```

    #10 point = 1;

    #10 point = 0;

    // 关闭

    #10 LE = 1;

    #20 $finish;

end

endmodule

```

[DispNum.v]

```

module DispNum(

    input  wire [1:0] BTN,      // 两个按钮: BTN[0]接 LE, BTN[1]接 point
    input  wire [7:0] SW,      // 8 个开关: 低 4 位数字, 高 4 位 AN 控制
    output wire [7:0] SEGMENT, // 数码管段选 (含小数点)
    output wire [3:0] AN,      // 数码管位选
    output wire      BTN_X     // 按钮行选, 恒为 0
);

    // 按钮行选恒为 0

    assign BTN_X = 1'b0;

    // 高 4 位开关直接控制位选

```

```
assign AN = SW[7:4];

// 内部连线

wire a, b, c, d, e, f, g, p;

// 实例化译码器

MyMC14495 u_decoder (

    .D0    (SW[0]),

    .D1    (SW[1]),

    .D2    (SW[2]),

    .D3    (SW[3]),

    .LE    (BTN[0]),    // LE 低有效

    .point(BTN[1]),    // point 高有效

    .p     (p),

    .a     (a),

    .b     (b),

    .c     (c),

    .d     (d),

    .e     (e),

    .f     (f),

    .g     (g)

);
```

```
// 拼接 SEGMENT 总线

// 按照要求: SEGMENT[7] = p

// SEGMENT[6] = g, SEGMENT[5] = f, SEGMENT[4] = e,

// SEGMENT[3] = d, SEGMENT[2] = c, SEGMENT[1] = b, SEGMENT[0] = a

assign SEGMENT = {p, g, f, e, d, c, b, a};

endmodule
```